

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1707

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 30%

QUESTIONS: 2

Duration : 45 Minutes
Total Marks:20

Annexure A

Descriptive Written Test

Code of Conduct:

I KISHORE KUMAR V(192521371)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

KISHORE

Annexure A

Descriptive Written Test

1. A smart home automation system is being designed to manage lighting, temperature, and security based on user behavior and environmental conditions. The system must respond to real-time inputs such as motion detection, temperature changes, and time of day while also learning user preferences over time. In this context, explain the different types of intelligent agents (simple reflex, model-based, goal-based, and utility-based agents) and analyze how each type would function within this system. Justify which type of agent or hybrid approach would be most effective for ensuring comfort, energy efficiency, and security.
2. A robot is placed in an unknown maze and must find a path to the exit without prior knowledge of the layout. Explain how uninformed search strategies like Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can help solve this problem.
Discuss the advantages and disadvantages of these algorithms in terms of time and space complexity. Relate the problem to different types of intelligent agents and explain how agent design affects decision-making. Suggest which combination of agent type and search strategy would be most effective and justify your choice.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

1. Intelligent Agents in a Smart Home Automation System

A **smart home automation system** receives inputs such as motion, temperature, time of day, and user preferences. An intelligent agent observes the environment through sensors and takes suitable actions through devices such as lights, fans, air conditioners, alarms, and locks.

The four important types of agents are **Simple Reflex Agent, Model-Based Agent, Goal-Based Agent, and Utility-Based Agent**.

1. Simple Reflex Agent

A **simple reflex agent** selects an action based only on the **current percept**. It uses condition-action rules.

Example:

- If motion is detected → turn ON the light.
- If temperature is above 28°C → turn ON the fan.
- If no motion is detected → turn OFF the light.

Working in smart home:

The agent continuously receives information from sensors and immediately responds according to predefined rules.

Advantages:

- Simple to design.
- Fast response.
- Easy to implement.

Disadvantages:

- Does not remember previous situations.
- Cannot learn user preferences.
- Cannot handle complex situations effectively.

2. Model-Based Agent

A **model-based agent** maintains an **internal model/state of the environment**. It can remember previous information and use it along with the current sensor input.

Example:

Suppose the system knows that:

- The user normally returns home at 6 PM.
- The living-room light was previously switched ON.
- The current temperature is 30°C.

The agent can use this information to decide whether to turn on the lights or adjust the temperature.

Working in smart home:

It maintains information about the state of rooms, devices, users, temperature, and security conditions.

Advantages:

- Maintains previous information.
- Works better in partially observable environments.
- Can make better decisions than a simple reflex agent.

Disadvantages:

- More complex than a simple reflex agent.

- Requires memory and an internal model.

3. Goal-Based Agent

A **goal-based agent** chooses actions that help it achieve a particular **goal**.

In a smart home, possible goals are:

- Maintain comfortable temperature.
- Reduce electricity consumption.
- Keep the house secure.
- Provide suitable lighting.

Example:

Goal: **Save energy**

The agent may:

1. Detect that nobody is in a room.
2. Check that the lights are ON.
3. Turn OFF unnecessary lights.
4. Keep essential security devices active.

The agent therefore selects actions based on whether they help achieve the desired goal.

Advantages:

- Can plan actions.
- Can handle different situations.
- More flexible than reflex agents.

Disadvantages:

- Requires more computation.
- Selecting the best action can take more time.

4. Utility-Based Agent

A **utility-based agent** not only tries to achieve a goal but also chooses the action that gives the **highest utility or overall benefit**.

For a smart home, utility can consider:

- Comfort
- Energy consumption
- Security
- Cost
- User preferences

Example:

Suppose the temperature is 30°C.

The agent could:

- Turn the AC ON at high power.
- Turn the fan ON.
- Set the AC to moderate cooling.

A utility-based agent compares the choices and selects the one that provides the best balance between **comfort and energy**

efficiency.

Comparison

Agent Type	Main Idea	Smart Home Example
Simple Reflex	Current condition → action	Motion detected → light ON
Model-Based	Uses current and previous state	Remembers room/device status
Goal-Based	Works toward a goal	Maintain comfortable temperature
Utility-Based	Chooses the best overall option	Comfort + low energy + security

Most Effective Approach

For a smart home, a **hybrid approach using model-based, goal-based, and utility-based agents** would be most effective.

A model-based component can maintain information about the home and user behavior. A goal-based component can define objectives such as comfort and security. A utility-based component can select the best action by balancing comfort, energy consumption, cost, and security.

For example, when the user enters the house, the system can recognize the user's normal behavior, turn on required lights, adjust the temperature, and avoid unnecessary energy consumption.

Conclusion

A simple reflex agent is useful for basic immediate actions, but it cannot handle complex smart-home requirements. A model-based agent provides memory, a goal-based agent provides planning, and a utility-based agent helps select the best action. Therefore, a **hybrid intelligent-agent system** provides the best solution for achieving **comfort, energy efficiency, and security**.

2. UCS and IDS for a Robot in an Unknown Maze

A robot placed in an unknown maze needs to find a path from its starting position to the exit. Since it has **no prior knowledge of the maze**, it can use **uninformed search algorithms**.

Two suitable strategies are:

1. **Uniform Cost Search (UCS)**
2. **Iterative Deepening Search (IDS)**

The question specifically asks to relate these algorithms to intelligent agents and compare their time and space complexity.

Uniform Cost Search (UCS)

Uniform Cost Search expands the node having the **lowest path cost** from the starting point.

It uses a priority queue.

Working

Suppose the robot starts at position **S** and wants to reach exit **G**.

UCS works as follows:

1. Start from the initial position.
2. Add the starting node to the priority queue.
3. Select the node with the lowest path cost.
4. Expand that node.
5. Add its unvisited neighboring positions to the queue.
6. Again select the lowest-cost node.

7. Continue until the goal is reached.

If every movement has the same cost, UCS behaves similarly to **Breadth-First Search (BFS)**.

Example

If there are three possible paths:

- Path A → cost 10
- Path B → cost 6
- Path C → cost 8

UCS selects **Path B first** because its current cost is the lowest.

Advantages of UCS

- Complete when step costs are positive.
- Finds the **least-cost path**.
- Useful when different movements have different costs.
- Does not require prior knowledge of the maze.

Disadvantages of UCS

- Can consume a large amount of memory.
- Can be slower when there are many paths.
- Stores many frontier nodes.

Complexity of UCS

Let:

- **b** = branching factor
- **C*** = cost of the optimal solution
- **ε** = minimum step cost

Approximate complexity:

Time:

$$O(b^{(1 + C^*/\epsilon)})$$

Space:

$$O(b^{(1 + C^*/\epsilon)})$$

Thus, UCS can require significant time and memory for a large maze.

Iterative Deepening Search (IDS)

Iterative Deepening Search combines the advantages of **Depth-First Search (DFS)** and **Breadth-First Search (BFS)**.

It repeatedly performs depth-limited DFS with increasing depth limits.

Working

For example:

Depth = 0

- Search only the starting node.

Depth = 1

- Search nodes one level away.

Depth = 2

- Search up to two levels.

Depth = 3

- Search up to three levels.

The process continues until the robot finds the exit.

Example

If the exit is at depth 4:

Limit 0 → Start

Limit 1 → Search up to level 1

Limit 2 → Search up to level 2

Limit 3 → Search up to level 3

Limit 4 → Find Exit

Advantages of IDS

- Uses much less memory than BFS.
- Complete when the branching factor is finite.
- Finds the shallowest solution when all step costs are equal.
- Suitable when the solution depth is unknown.

Disadvantages of IDS

- Repeats the search from the beginning at each depth.
- This causes additional computation.
- It is not suitable for different path costs because it considers depth rather than actual cost.

Complexity of IDS

Let:

- **b** = branching factor
- **d** = depth of the shallowest solution

Then:

Time:

$O(b^d)$

Space:

$O(bd)$

Therefore, IDS has significantly lower memory requirements than UCS in many cases.

UCS vs IDS

Feature	UCS	IDS
Search basis	Path cost	Depth
Memory	High	Low
Time	Can be high	$O(b^d)$
Space	High	$O(bd)$
Complete	Yes, with positive costs	Yes, for finite branching
Optimal	Yes, with suitable positive costs	Yes, when all step costs are equal

Feature	UCS	IDS
Best use	Different movement costs	Unknown solution depth
Main drawback	High memory usage	Repeated searching

Relation to Intelligent Agents

The robot can be considered an **intelligent agent** because it:

1. Observes its environment using sensors.
2. Receives information about walls and available paths.
3. Selects actions based on the available information.
4. Moves toward the goal.
5. Updates its knowledge as it explores the maze.

Simple Reflex Agent

A simple reflex agent can react to immediate conditions.

Example:

If wall ahead → turn right.

However, this approach may fail to find the best path because it does not remember previous states.

Model-Based Agent

A model-based agent can maintain an internal representation of the maze.

For example, it can remember:

- Visited locations.
- Blocked paths.
- Available paths.
- Current position.

This is more suitable for an unknown maze.

Goal-Based Agent

A goal-based agent focuses on reaching the **exit**.

It evaluates possible movements based on whether they help reach the goal.

Utility-Based Agent

A utility-based agent can choose a path based on several factors, such as:

- Shortest distance.
- Lowest movement cost.
- Safety.
- Battery consumption.

Therefore, a utility-based approach can be useful when different paths have different costs.

Best Combination for the Maze

For an unknown maze, a **model-based or goal-based agent combined with an appropriate uninformed search algorithm** is effective.

If all movements have equal cost:

Model-Based Agent + IDS

is a good choice because IDS:

- Requires less memory.
- Can find a shallow solution.
- Does not require prior knowledge of the maze.
- Works well when the depth of the exit is unknown.

If movement costs are different:

Model-Based Agent + UCS

is preferable because UCS considers the actual path cost and can find the **least-cost path**.

Final Recommendation

For a normal maze where every movement has the same cost, **Model-Based Agent + IDS** is a suitable combination.

The model-based agent remembers the explored maze, while IDS searches progressively deeper until it finds the exit. If different paths have different costs, **UCS** should be preferred because it guarantees the least-cost solution under its usual positive-cost assumptions.